

# Security Audit

Ulalo (ERC-20 Token)

# Table of Contents

|                                   |    |
|-----------------------------------|----|
| Executive Summary                 | 4  |
| Project Context                   | 4  |
| Audit Scope                       | 7  |
| Security Rating                   | 8  |
| Intended Smart Contract Functions | 9  |
| Code Quality                      | 10 |
| Audit Resources                   | 10 |
| Dependencies                      | 10 |
| Severity Definitions              | 11 |
| Status Definitions                | 12 |
| Audit Findings                    | 13 |
| Centralisation                    | 21 |
| Conclusion                        | 22 |
| Our Methodology                   | 23 |
| Disclaimers                       | 25 |
| About Hashlock                    | 26 |

## CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

## Executive Summary

The Ulalo team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

## Project Context

ULALO is an emerging health-tech blockchain initiative designed to revolutionize the medical sector by incentivizing content creators and influencers to promote its mission. Through its Ambassador Program, ULALO empowers passionate individuals to spread awareness about the platform's advancements while being rewarded with \$ULA tokens, exclusive NFTs, and early access to platform updates. This approach not only builds a global community but also supports ULALO's goal of integrating blockchain-based solutions into healthcare communication and outreach efforts

**Project Name:** Ulalo

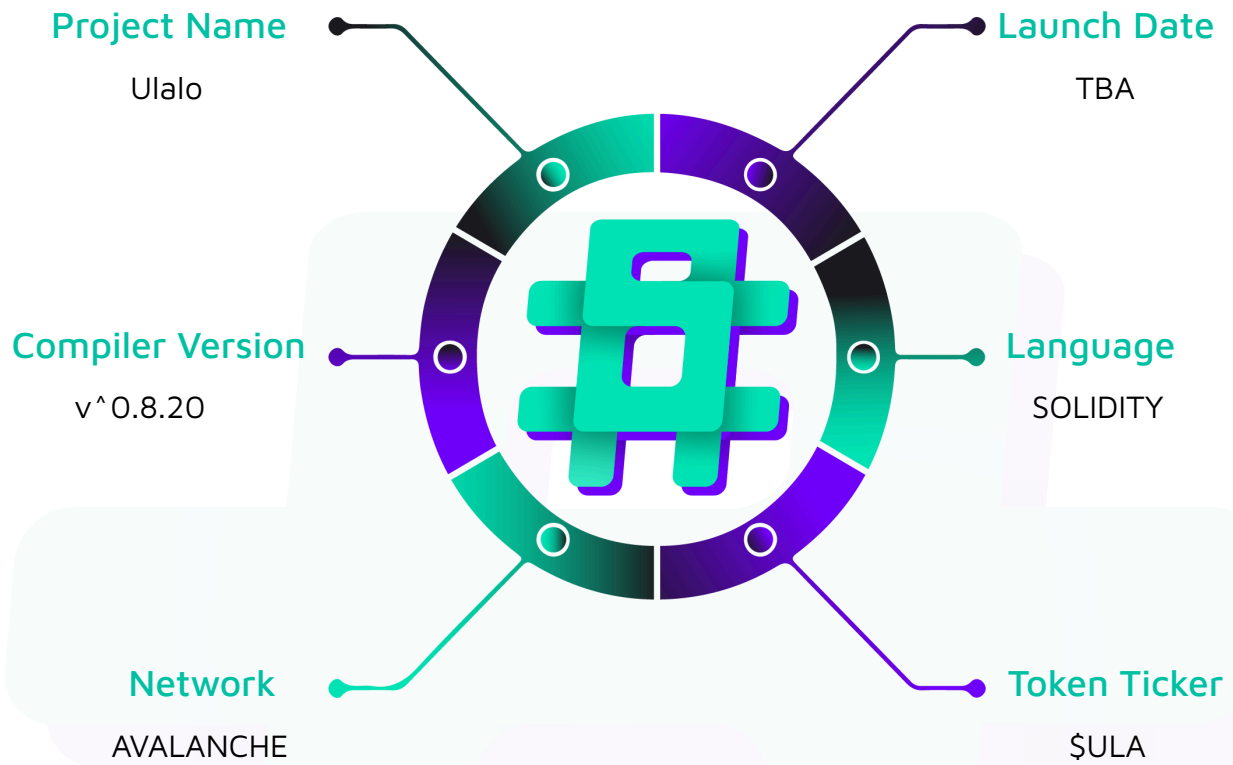
**Project Type:** ERC-20 Token

**Compiler Version:** ^0.8.20

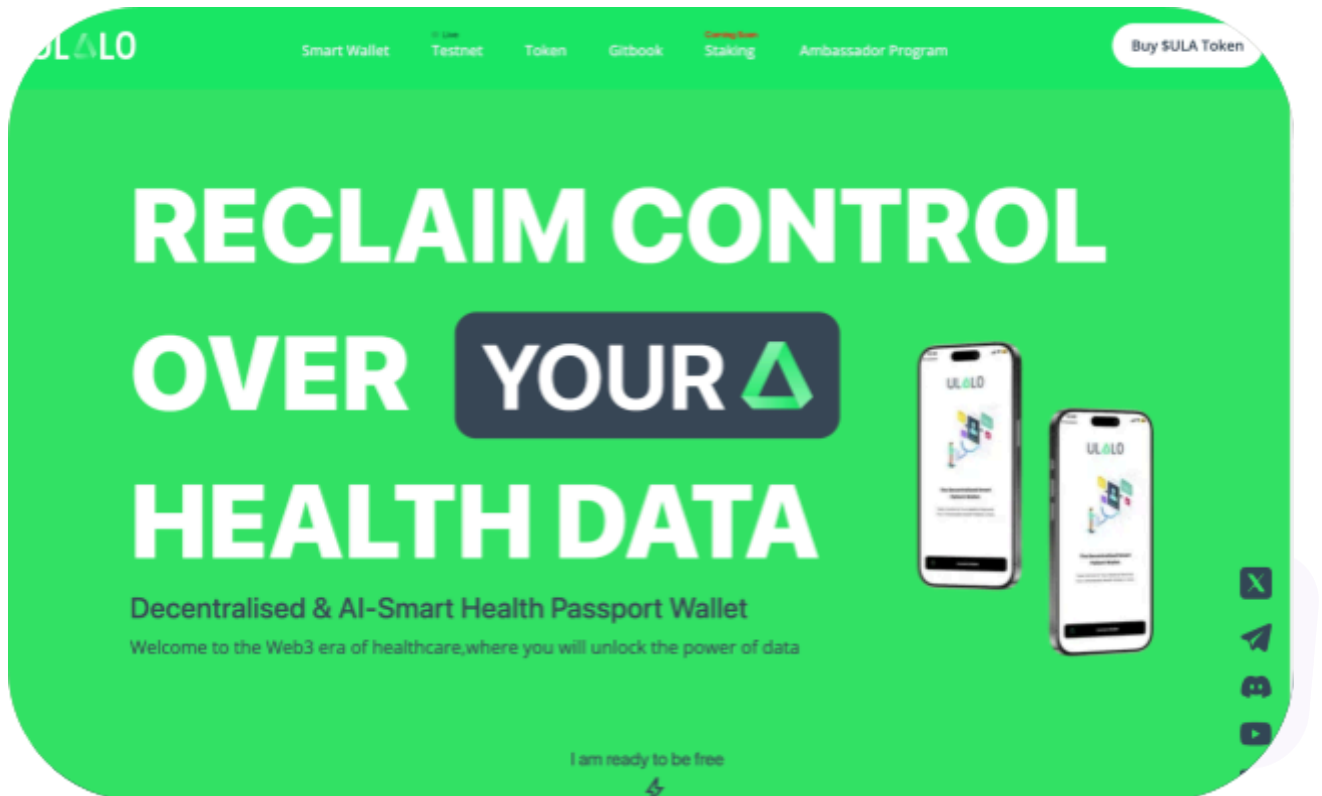
**Website:** <https://www.ulalo.xyz/>

**Logo:**

The logo for ULALO, featuring the word "ULALO" in a bold, dark blue, sans-serif font. The letter "A" is replaced by a green triangle pointing upwards.

**Visualised Context:**

## Project Visuals:



## Audit Scope

We at Hashlock audited the solidity code within the Ulalo project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

| Description            | Ulalo Smart Contracts                      |
|------------------------|--------------------------------------------|
| Platform               | Avalanche / Solidity                       |
| Audit Date             | June, 2025                                 |
| Contract 1             | UlaloToken.sol                             |
| Contract 1 MD5 Hash    | d215f44138a69b185b020a4044f194c0           |
| Contract 1 Address     | 0xf936F99D28470A53a315873087c6Ea21bb246912 |
| Fix Review Commit Hash | 9e6e733b7acd04ce5d3b10719227df2d8d9edc2d   |

# Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

*The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.*

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

## Hashlock found:

1 Medium severity vulnerability

5 Low severity vulnerabilities

**Caution:** Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.



## Intended Smart Contract Functions

| Claimed Behaviour                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Actual Behaviour                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| <p><b>UlaloToken.sol</b></p> <p>Allows users to:</p> <ul style="list-style-type: none"> <li>- Transfer tokens to other addresses</li> <li>- Burn their own tokens</li> <li>- Be protected by rate limiting and cooldown periods on transfers</li> </ul> <p>Allows admins to:</p> <ul style="list-style-type: none"> <li>- Mint new tokens (MINTER_ROLE)</li> <li>- Burn tokens from other accounts(BURNER_ROLE)</li> <li>- Pause all token transfers (PAUSER_ROLE)</li> <li>- Blacklist addresses (BLACKLISTER_ROLE)</li> <li>- Recover ERC20 tokens accidentally sent to the contract</li> <li>- Set transfer limit percentage</li> <li>- Set transfer cooldown period</li> </ul> | <p>Contract achieves this functionality.</p> |

## Code Quality

This audit scope involves the smart contracts of the Ulalo project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

## Audit Resources

We were given the Ulalo project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

## Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

## Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

| Significance | Description                                                                                                                                                                                           |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| High         | High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community. |
| Medium       | Medium-level difficulties should be solved before deployment, but won't result in loss of funds.                                                                                                      |
| Low          | Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.                                                                                        |
| Gas          | Gas Optimisations, issues, and inefficiencies.                                                                                                                                                        |
| QA           | Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.                            |

## Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

| Significance        | Description                                                                                                                                                                                                                                                                                                          |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Resolved</b>     | The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.                                                                                   |
| <b>Acknowledged</b> | The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception. |
| <b>Unresolved</b>   | The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.                                                                                                                                                                                               |

# Audit Findings

## Medium

### [M-01] UlaloToken - Centralized Role Management Creates Single Point of Failure in Authorization System

#### Description

The UlaloToken contract utilizes OpenZeppelin's AccessControl for role-based permissions, but assigns all critical roles to a single initialOwner address in the constructor. This design creates a centralized authorization structure with a single point of failure that threatens the entire protocol's security.

#### Vulnerability Details

The constructor grants all privileged roles to the same address:

```
constructor(  
    string memory name,  
    string memory symbol,  
    address initialOwner  
) ERC20(name, symbol) {  
    _grantRole(DEFAULT_ADMIN_ROLE, initialOwner);  
    _grantRole(MINTER_ROLE, initialOwner);  
    _grantRole(BURNER_ROLE, initialOwner);  
    _grantRole(PAUSER_ROLE, initialOwner);  
    _grantRole(BLACKLISTER_ROLE, initialOwner);  
  
    _mint(initialOwner, 100000000 * 10 ** decimals());  
}
```

```
}
```

This approach violates the principle of separation of duties, which is a fundamental security concept. While the `DEFAULT_ADMIN_ROLE` can later reassign roles to different addresses, the initial centralized setup creates a significant vulnerability window during the critical early period of the protocol's lifecycle.

## Impact

If the `initialOwner` private key is compromised, an attacker would gain:

1. Unlimited minting capabilities (`MINTER_ROLE`)
2. Ability to burn anyone's tokens (`BURNER_ROLE`)
3. Power to pause the entire protocol (`PAUSER_ROLE`)
4. Control over blacklisting any user (`BLACKLISTER_ROLE`)
5. Administrative access to modify any protocol parameter (`DEFAULT_ADMIN_ROLE`)

This represents a catastrophic security risk that could lead to complete economic manipulation of the token, denial of service for legitimate users, and potentially permanent damage to the protocol's reputation and functionality.

## Recommendation

Implement a more distributed role management system from the outset:

```
constructor(
    string memory name,
    string memory symbol,
    - address initialOwner
    + address admin,
    + address minter,
    + address burner,
    + address pauser,
    + address blacklister
) ERC20(name, symbol) {
```

```
// Setup roles with separate addresses

- _grantRole(DEFAULT_ADMIN_ROLE, initialOwner);
- _grantRole(MINTER_ROLE, initialOwner);
- _grantRole(BURNER_ROLE, initialOwner);
- _grantRole(PAUSER_ROLE, initialOwner);
- _grantRole(BLACKLISTER_ROLE, initialOwner);

+ _grantRole(DEFAULT_ADMIN_ROLE, admin);
+ _grantRole(MINTER_ROLE, minter);
+ _grantRole(BURNER_ROLE, burner);
+ _grantRole(PAUSER_ROLE, pauser);
+ _grantRole(BLACKLISTER_ROLE, blacklister);


// Initialize with a fixed supply

- _mint(initialOwner, 100000000 * 10 ** decimals());
+ _mint(admin, 100000000 * 10 ** decimals());
}
```

## Status

Resolved

## Low

### [L-01] UlaloToken - Inconsistent Pausability Implementation Allows Partial Contract Functionality When Paused

#### Description

The UlaloToken contract inherits from OpenZeppelin's Pausable but fails to apply the whenNotPaused modifier consistently across all external functions. Currently, the contract only applies the modifier to the internal \_update function that handles token transfers:

```
function _update(  
    address from,  
    address to,  
    uint256 amount  
) internal virtual override whenNotPaused nonReentrant {  
    // Implementation details  
}
```

While this does effectively prevent token transfers, minting, and burning operations when the contract is paused (as these all call \_update), several other external functions remain callable:

- approve (inherited from ERC20)
- setTransferLimitPercentage
- setTransferCooldown
- updateBlacklist
- recoverERC20

This creates an inconsistent state where the contract is technically "paused" but still allows significant functionality.



## Recommendation

Apply the `whenNotPaused` modifier consistently to all user-facing functions that should be disabled during a pause state.

## Status

Resolved

### [L-02] UlaloToken - Missing Pausable State Validation in `pause()/unpause()` Functions Allows Redundant Pausing Operations

#### Description

The `UlaloToken` contract implements `pause()` and `unpause()` functions that allow users with the `PAUSER_ROLE` to toggle the contract's operational state. However, both implementations fail to validate whether the contract is already in the target state before calling the internal OpenZeppelin functions.

According to OpenZeppelin's documentation for the `Pausable` contract, the internal functions have explicit requirements:

- `_pause()`: "The contract must not be paused"
- `_unpause()`: "The contract must be paused"

By failing to check these conditions, both functions could be called redundantly when the contract is already in the target state, violating the intended design of the `Pausable` abstraction.

While OpenZeppelin's implementations include checks that will revert if the conditions aren't met, relying on these internal checks instead of handling them explicitly in the public-facing functions is an implementation weakness. It creates unnecessary gas costs for failed transactions and doesn't properly encapsulate the expected behavior at the appropriate abstraction level.

## Recommendation

Add explicit checks in both functions to verify the current pause state before attempting to modify it.

## Status

Resolved

### [L-03] UlaloToken - Unsafe ERC20 Transfer in recoverERC20 Function Allows Silent Failures

## Description

The UlaloToken contract implements a recoverERC20 function that allows the admin to recover ERC20 tokens accidentally sent to the contract. However, the function uses the standard transfer method instead of safeTransfer from OpenZeppelin's SafeERC20 library:

```
function recoverERC20(
    address tokenAddress,
    uint256 tokenAmount
) external override onlyRole(DEFAULT_ADMIN_ROLE) {
    // Ensure we're not trying to recover this token
    require(
        tokenAddress != address(this),
        "UlaloToken: Cannot recover the token itself"
    );

    IERC20(tokenAddress).transfer(msg.sender, tokenAmount);
    emit TokenRecovered(tokenAddress, tokenAmount);
}
```

Using the standard `transfer` method is problematic because some ERC20 tokens don't conform to the standard return value pattern. Notable examples include USDT, which doesn't return a boolean value, and BNB, which returns `false` on failure instead of reverting. When interacting with such non-compliant tokens, the `transfer` call may fail silently, leading the admin to believe tokens were recovered when they weren't.

Even though this function is restricted to admin users only, the potential for silent failures creates an operational risk where the admin might incorrectly assume a token recovery was successful.

### Recommendation

Replace the standard `transfer` call with `safeTransfer` from OpenZeppelin's SafeERC20 library to ensure proper handling of all ERC20 token implementations.

### Status

Resolved

## [L-04] UlaIoToken - Inconsistent Event Emission in Token Operations

### Description

The UlaIoToken contract implements several token manipulation functions but lacks consistent event emission patterns. Specifically, there are multiple instances where important token state changes occur without corresponding events being emitted:

1. In the constructor, a significant amount of tokens are minted to the initial owner without emitting any event.
2. The `mint` function doesn't emit a `Minted` event (or equivalent) after minting tokens.
3. The `burn` function doesn't emit a `TokensBurned` event, while its counterpart `burnFrom` does.

This inconsistent pattern of event emission reduces the contract's off-chain observability, making it difficult for external systems to reliably track and react to token state changes. While the ERC20 base contract does emit `Transfer` events for all

transfers (including mints and burns), custom events provide additional context that is valuable for tracking specific operation types.

### Recommendation

Add appropriate event emissions to all functions that modify token state to ensure consistent and complete off-chain observability.

### Status

Resolved

## [L-05] UlaloToken - Missing Zero-Address Validation

### Description

The UlaloToken contract fails to validate against zero-address input in several functions, creating the potential for unrecoverable token loss. This validation is missing in the following key functions:

1. The constructor lacks zero-address validation for the `initialOwner` parameter.
2. The `mint` function doesn't validate that the recipient address is non-zero.
3. The `updateBlacklist` function doesn't check if the target address is the zero address.

While the OpenZeppelin ERC20 implementation does include some internal checks, explicitly validating zero addresses at the public function level provides an additional layer of protection against user error and follows security best practices.

### Recommendation

Add zero-address validation to all functions that accept address parameters.

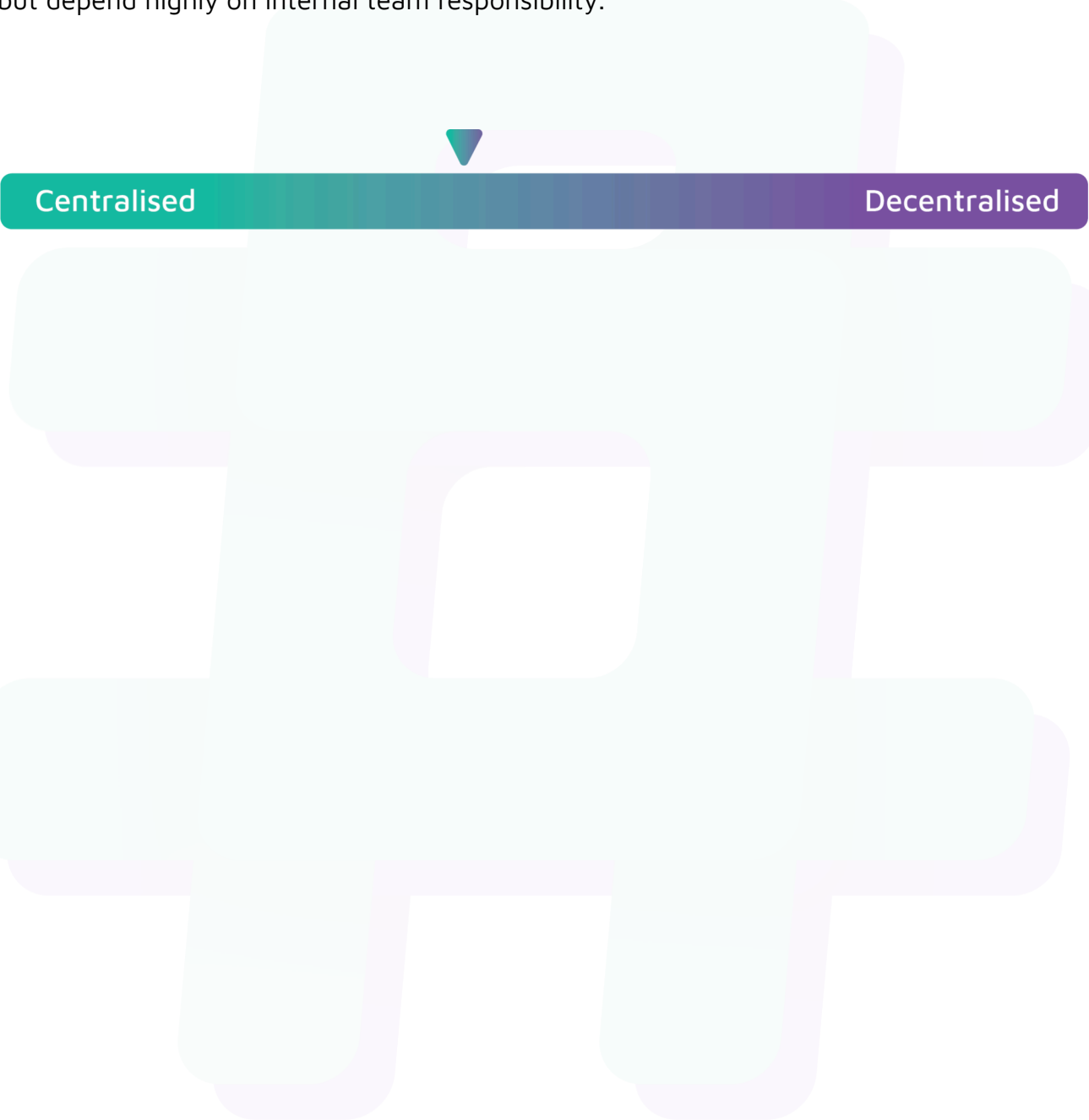
### Status

Resolved

## Centralisation

The Ulalo project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

## Conclusion

After Hashlock's analysis, the Ulalo project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

## Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

### **Manual Code Review:**

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

### **Vulnerability Analysis:**

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

**Documenting Results:**

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

**Suggested Solutions:**

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.



# Disclaimers

## Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

## Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

## About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

**Website:** [hashlock.com.au](https://hashlock.com.au)

**Contact:** [info@hashlock.com.au](mailto:info@hashlock.com.au)



**#hashlock.**